

Tecnicatura:

**TECNICATURA SUPERIOR EN CIENCIA DE
DATOS E INTELIGENCIA ARTIFICIAL**



|Segundo Cuatrimestre - 2do Año

|MÓDULO - Práctica PROFESIONALIZANTE I

|Docente - ACCIETTO Dianiel

|Grupo - N²-EFMG Solutions

Cliente: Escuela PRoA (Río Tercero)

Desarrollador: N²-EFMG

Fecha: Septiembre de 2026

Integrantes & Roles:

- Valeria Nieves Villalba (Business Analyst)
- Florencia Santarena (Data Scientist / Backend)
- Mariela Suarez (Machine Learning Engineer)
- Juan Gabriel Cabrera (Full Stack QA Engineer)
- Nicolás Allende Olmedo (Project Manager)
- Emanuel Guaraz (IA Engineer)

INTRODUCCIÓN Y CONTEXTO DEL PROYECTO

El presente informe técnico detalla la evolución de la solución desarrollada por la empresa **N²-EFMG Solutions** para la **Escuela PROA Sede Río Tercero**. En la fase inicial del Proyecto Integrador, se identificó un punto de dolor operativo crítico: la comunidad educativa y los entes locales carecían de una infraestructura centralizada y automatizada para almacenar e historiar las variables climáticas y ambientales de su **Estación Meteorológica y Ambiental Conceptual** (basada en microcontroladores Arduino UNO). Esto obligaba a los usuarios a registrar los datos de forma manual desde el Monitor Serie o a perderlos al desconectar el dispositivo, imposibilitando el análisis estadístico, el estudio de series temporales y la generación de métricas ambientales representativas para la ciudad y zonas aledañas.

Para mitigar este problema, se implementó exitosamente una canalización de ingesta robusta mediante un script en Python que captura los datos del puerto serie vía USB y los persiste de manera estructurada en una base de datos MySQL local (con soporte de un buffer offline en formato CSV ante caídas de conexión). No obstante, para que esta valiosa base de datos se traduzca en decisiones inteligentes y accesibles en tiempo real —tanto para directivos, docentes y estudiantes, como para entes municipales e industriales que sufren la falta de microdatos meteorológicos y de calidad ambiental precisos en el departamento Tercero Arriba—, se requiere una interfaz de usuario accesible y sin fricción. Es así como surge el diseño e implementación de un **Chatbot de WhatsApp**, orientado a la consulta inmediata y democratización de los datos ambientales en Río Tercero y la región.

Definición de la Solución de Fase 2: El Chatbot de WhatsApp actuará como una interfaz interactiva de consulta. Cualquier usuario autorizado podrá enviar mensajes predefinidos para obtener de forma instantánea el 'Clima' (lectura en tiempo real), el 'Estado del Tiempo' (tendencia reciente) o un 'Reporte' consolidado del día. Además, el sistema integrará un servicio demonio proactivo encargado de lanzar alertas críticas inmediatas vía WhatsApp cuando la temperatura registrada en la base de datos supere los 32°C o sea inferior a 8°C, salvaguardando así las actividades del aula e invernaderos.

01. DOCUMENTO DE REQUERIMIENTOS Y TRAZABILIDAD

De acuerdo con los lineamientos establecidos en el documento de **Próximos Pasos**, cada requerimiento funcional debe estar trazado formalmente a las familias de puntos de dolor (Pain Points) detectados originalmente. A continuación, se presenta la matriz de requerimientos del sistema de visualización y alertas a través de WhatsApp.

ID	Familia Pain Point	Requerimiento Técnico Funcional (RF) / No Funcional (RNF)	Descripción del Requerimiento
RF-01	PP-04 Experiencia (Falta de Visibilidad)	Consulta de Clima en Tiempo Real	El Chatbot debe responder de manera instantánea ante las palabras clave 'Clima' o 'Actual', retornando el último registro de temperatura persistido en MySQL, junto con la fecha y hora del sensor.

RF-02	PP-03 Decisión (Decisión sin Sustento)	Análisis de Tendencia (Estado del Tiempo)	El Chatbot debe responder a la consulta 'Estado del Tiempo' calculando la pendiente de temperatura de la última hora en la base de datos, informando si el clima se encuentra en aumento, estable o en descenso.
RF-03	PP-02 Datos (Datos dispersos/sucios)	Consolidación de 'Reporte' Diario	Al recibir la palabra 'Reporte', el Chatbot ejecutará un análisis agregativo en MySQL (utilizando Pandas en backend) y devolverá las temperaturas máxima, mínima y promedio de las últimas 24 horas.
RNF-01	PP-04 Experiencia (Visibilidad)	Latencia de Alertas Corta (<60 segundos)	El tiempo transcurrido desde que un sensor Arduino escribe una temperatura fuera de rango en la base de datos MySQL hasta el envío del mensaje de alerta en WhatsApp no debe superar los 60 segundos.
Desarrollo de Alertas Tempranas: (Para Práctica Profesionalizante II)			
RNF-02	PP-01 Proceso (Fricción Operativa)	Disponibilidad Continua (24/7)	La API Webhook del Chatbot y el daemon de alertas deben ejecutarse de manera ininterrumpida con un uptime objetivo del 99.5%.
RF-04	PP-04 Experiencia / PP-01 (Fricción y Alertas)	Alerta de Temperatura Crítica Alta (>32°C)	Un servicio demonio en Python debe escanear la base de datos. Si la temperatura ingresada es superior a 32°C, se disparará automáticamente una alerta masiva al grupo de WhatsApp establecido.
RF-05	PP-04 Experiencia / PP-01 (Fricción y Alertas)	Alerta de Temperatura Crítica Baja (<8°C)	Si el escaneo de la base de datos registra una lectura de temperatura inferior a 8°C, el demonio emitirá una notificación de alerta crítica de helada o frío extremo al canal de WhatsApp.

CRITERIOS DE ACEPTACIÓN

Para validar el correcto funcionamiento del Chatbot y del sistema de alertas meteorológicas, se definen los siguientes escenarios de prueba y criterios de aceptación específicos:

Escenario: Consulta de Clima Actual (RF-01)	
Dado que...	La base de datos MySQL tiene un último registro persistido de: sensor_id='Arduino_Aulas', temperatura=23.4, fecha_hora='2026-09-02 15:45:00'.
Cuando...	Un usuario registrado envía la palabra 'Clima' o 'Actual' al chatbot de WhatsApp.
Entonces...	El chatbot consulta la base de datos, extrae el registro en menos de 2 segundos, y responde: <i>'Estación PProA: Temp. Actual es de 23.4°C (Leído a las 15:45:00).'</i>

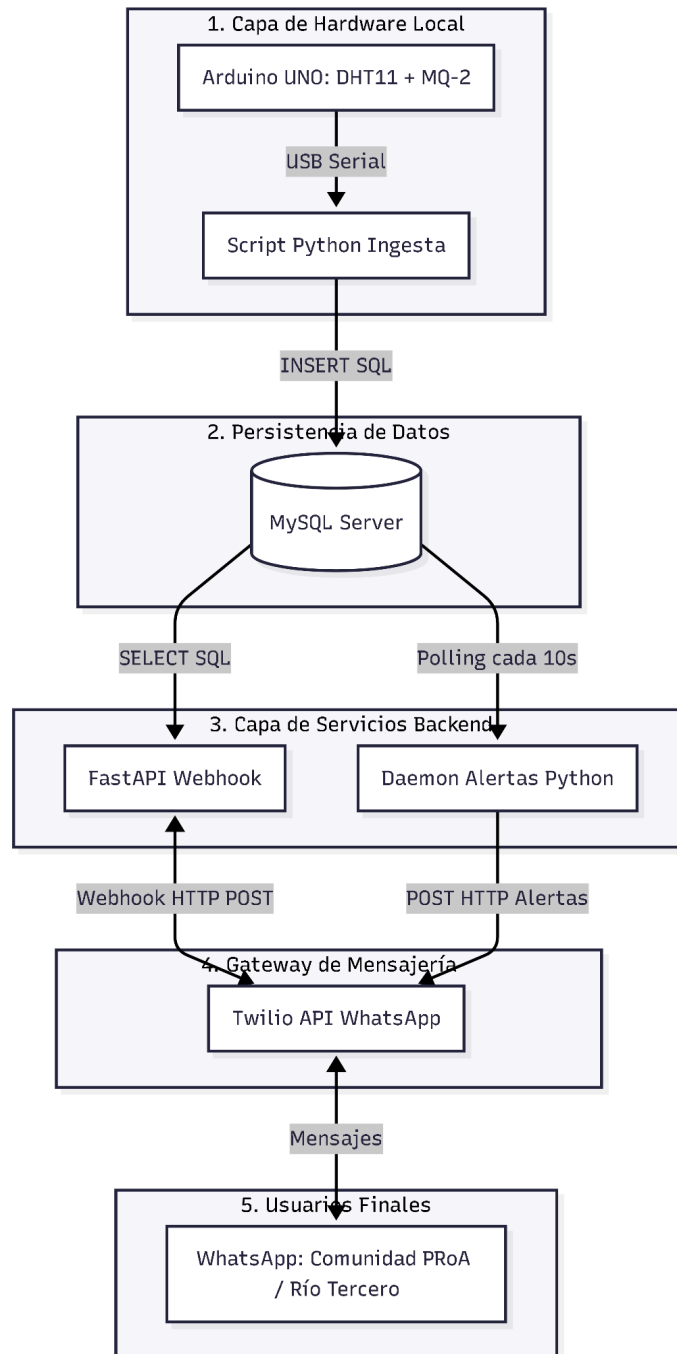
Desarrollo de Alertas Tempranas: (Para Práctica Profesionalizante II)

Escenario: Alerta Proactiva de Temperatura Crítica Alta (RF-04)	
Dado que...	El sensor Arduino registra un pico de calor en el invernadero de 32.5°C y lo inserta automáticamente en la tabla MySQL.
Cuando...	El demonio de monitoreo de alertas realiza su escaneo periódico de 10 segundos y detecta un valor superior a 32°C.
Entonces...	El demonio envía una petición POST HTTP a Twilio, la cual redirige en menos de 10 segundos una notificación crítica a los contactos del colegio PProA diciendo: <i>'ALERTA DE CALOR: Temperatura crítica superada en Escuela PProA Río Tercero: 32.5°C. Tomar precauciones de ventilación.'</i> El sistema bloquea alertas repetitivas por los próximos 30 minutos.

Escenario: Alerta Proactiva de Temperatura Crítica Baja (RF-05)	
Dado que...	La temperatura del entorno desciende a 7.5°C debido a una helada invernal nocturna, registrándose en MySQL.
Cuando...	El servicio demonio detecta en su lectura recurrente que el valor es inferior a 8°C.
Entonces...	El demonio dispara de inmediato una notificación push de WhatsApp indicando: <i>'ALERTA DE HELADA: Temperatura crítica baja en Escuela PProA Río Tercero: 7.5°C. Activar protocolos de protección térmica.'</i>

02. ARQUITECTURA DEL SISTEMA Y FLUJO DE DATOS

El esqueleto técnico de la solución se ha estructurado siguiendo el principio de *Simplicidad* expresado en el documento de *Próximos Pasos*, utilizando un acoplamiento laxo entre los módulos para garantizar escalabilidad, facilidad de mantenimiento y desacoplamiento de la base de datos



EXPLICACIÓN DE LOS FLUJOS DE DATOS

Descripción Técnica de la Arquitectura:

1. Capa de Captura y Adquisición Fija (Hardware e Ingesta):
Los sensores (DHT11 y MQ-2) conectados al microcontrolador Arduino UNO toman lecturas periódicas de temperatura, humedad y gases ambientales. Estos datos son transmitidos mediante comunicación serie sobre USB hacia un script local en Python (HU-04a), el cual se encarga del pareo, validación y control de errores en la trama recibida.
2. Capa de Persistencia Relacional:
El script de ingesta ejecuta sentencias `INSERT` parametrizadas para almacenar las lecturas estructuradas con marca

temporal (TIMESTAMP) en la base de datos MySQL. En caso de pérdida de conectividad con el motor relacional, el sistema activa un buffer secundario en formato CSV (HU-04b) garantizando cero pérdida de datos históricos.

3. Capa de Servicios Backend y Lógica de Negocio:

Se divide en dos componentes desacoplados para garantizar alta disponibilidad y escalabilidad:

- Servicio Webhook (FastAPI): Atiende el Flujo Reactivo Síncrono. Recibe las peticiones HTTP POST enviadas por la pasarela de WhatsApp cuando un usuario consulta comandos como Clima o Reporte, ejecutando consultas optimizadas de lectura (SELECT) a la base de datos MySQL y respondiendo en milisegundos.
- Servicio Demonio de Alertas (Daemon Python): Atiende el Flujo Proactivo Asíncrono. Realiza un escaneo recurrente (polling cada 10 segundos) sobre el último registro en MySQL. Si identifica valores fuera de los rangos operativos (por ejemplo, temperaturas > 32° o < 8°, verifica el registro de cool-down (evitando duplicados en 30 minutos) e inicia la notificación.

4. Capa de Pasarela (Gateway) e Interfaz de Usuario:

La pasarela Twilio API para WhatsApp actúa como puente seguro entre la infraestructura local backend y los dispositivos móviles. Transmite tanto las respuestas a las consultas directas como la emisión masiva de alertas críticas directamente hacia la comunidad educativa de la Escuela PRoA Río Tercero y los actores municipales o regionales.

Tabla de Stack Tecnológico:

Capa / Categoría	Componente Seleccionado	Función en el Sistema
Lenguaje Base	Python 3.12	Lenguaje principal para la lógica de ingesta serial, el servicio Webhook (FastAPI) y el demonio de alertas de fondo.
Backend Web / Webhook	FastAPI	Framework REST asíncrono para la recepción de Webhooks HTTP POST de Twilio, el procesamiento de comandos y el retorno acelerado de respuestas.
Pasarela de Mensajería	Twilio API para WhatsApp	Gateway oficial de comunicación que desacopla la infraestructura local y permite el intercambio seguro de mensajes con los usuarios.
Base de Datos	MySQL (Server local)	Motor relacional encargado de la persistencia centralizada e historización de las series temporales capturadas por el hardware.
Librerías Clave	<code>mysql-connector-python</code> , <code>pandas</code> , <code>colorama</code>	Gestión de la conectividad SQL, análisis estadístico de promedios/tendencias térmicas y formateo visual de la consola local.
Despliegue y Orquestación	Docker & <code>systemd</code>	Contenedorización modular de la API Webhook y ejecución ininterrumpida del demonio de monitoreo como servicio del sistema (24/7).

03. JUSTIFICACIÓN DE DECISIONES TÉCNICAS Y DESCARTES

En concordancia con el requerimiento de la *Evidencia de Aprendizaje 1* de justificar brevemente el diseño e informar sobre alternativas descartadas, se documentan las siguientes decisiones clave:

1- Decisión Key: Canal WhatsApp elegido vs. Desarrollo de una App Web o Móvil Propia: Se seleccionó WhatsApp como la interfaz del usuario final por su nula fricción de adopción. Desarrollar una aplicación nativa o web de monitoreo meteorológico requiere que los usuarios descarguen una app o naveguen activamente a un sitio, generando bajas tasas de retención. WhatsApp, siendo una herramienta de uso cotidiano, asegura que tanto docentes, estudiantes como los productores locales de Río Tercero tengan acceso directo y natural a los reportes sin curva de aprendizaje.

2- Decisión Key: FastAPI vs. Flask para el Webhook: Se optó por FastAPI debido a su soporte nativo para programación asíncrona (async/await), lo cual es crítico al manejar solicitudes Webhook de red concurrentes de la API de WhatsApp, logrando tiempos de respuesta de milisegundos y un consumo mínimo de hardware en la infraestructura local de la escuela. Se descartó Flask por su naturaleza síncrona por defecto, lo que podría generar cuellos de botella ante múltiples consultas simultáneas.

3- Decisión Key: Daemon de Alertas en Python vs. Triggers/Disparadores en MySQL: Se determinó programar un servicio de fondo ('demonio') en Python independiente que escanee la base de datos MySQL en lugar de utilizar disparadores (triggers) SQL a nivel de motor. Un trigger en MySQL forzará al motor de base de datos a llamar a una función externa (UDF) para invocar APIs web externas (como Twilio), lo cual degrada severamente el rendimiento de la persistencia de datos y acopla de manera peligrosa la lógica de negocio a la base de datos. El demonio en Python mantiene la separación de responsabilidades intacta y facilita la gestión de reintentos e intervalos de gracia.

CONCLUSIÓN Y PRÓXIMOS PASOS DE CONSTRUCCIÓN

Esta propuesta arquitectónica y de requerimientos consolida un sistema inteligente, unificando el hardware existente en la Escuela PRoA Río Tercero con un flujo moderno de datos e interfaces de nula fricción. El chatbot de WhatsApp transforma los datos climáticos crudos en respuestas entendibles y alertas proactivas vitales para la comunidad educativa y local, cumpliendo íntegramente con la visión de **N2-EFMG Solutions**: *“Transformando el ruido de los sensores en decisiones inteligentes.”*

Equipo de Ingeniería de Datos & QA:

- Valeria N. Villalba (Business Analyst)
- Florencia Santarena (Data / Backend)
- Nicolás Allende Olmedo *Project Manager (PM)*
- Mariela Suárez (ML Engineer)
- Juan G. Cabrera (QA Engineer)
- Emanuel Guaráz (IA Engineer)